Implicit Surfaces

From a eld function f : R ^{3} ! R over the domain of the terrain you can implicity de ne a surface the following way:

$$S = fp\ 2\ R\ ^3 jf(p) = 0g \tag{2.3}$$

That is, the surface would contain the set of all points which map to 0 in the scalar eld f. Going from a height eld h to an implicit surface is really straight-forward (although the opposite is not), you just need to de ne eld f as f(p) = h(p $_{xy}$) p $_z$, from that you can go to a usual volumetric representation, de ning as (p) = 1 if f(p) < 0 and (p) = 0 otherwise (meaning the material will be rock if and only if the function f is negative) for example.

Scalar elds de ned from elevation functions are a type of distance elds: for every point of the 3D space the eld represents the vertical distance to the surface, being negative if the point is inside and positive otherwise. You can also use other distance elds to de ne surfaces (with any de nition of distance that ts your situation) and you could easily transform those distance elds into standard volumetric models by assigning di erent distances to di erent materials.

2.2.2 Terrain Synthesis

Terrain generation techniques can be divided in three main groups (although it is common to employ methods from the three groups in the same project):

- Procedural Generation: Procedural generation methods arti cially generate terrains from scratch by applying randomized processes, like faulting or using noise.
- Example-based: Example-based methods create virtual terrains using scanned-data of real-world terrains.
- Erosion Simulation: Erosion simulations methods apply algorithms to simulate geomorphological processes when creating the landscape.

Synthetic terrains usually can not represent all the complexity of a natural landscape by procedural generation alone, and that is why erosion simulations are so important, not only in order to "predict" a future for the landscape but also to add realism on virtual scenes. Even example-based terrains often fail to capture some aspects of the terrain they are trying to replicate, as scans have precision limitations (and the bigger the region to scan, the more di cult it is to do it with accuracy), and erosion simulation can also intervene in those cases to add more complexity to the landscapes.

2.3 Model Decompositions

One of the core aspects of simulating erosion on 3D terrains is the model representation that we use. Any erosion simulation applied on a 3D model will modify the model and its properties, so proper data structures that allow for dynamic modi cations of the model are key for these types of simulations. Additionally, applying erosion to an object also exposes the interior of said object, so any model representation that we use also has to encode some internal information of the model, as internal points of a model might become part of the surface at some point of the process.

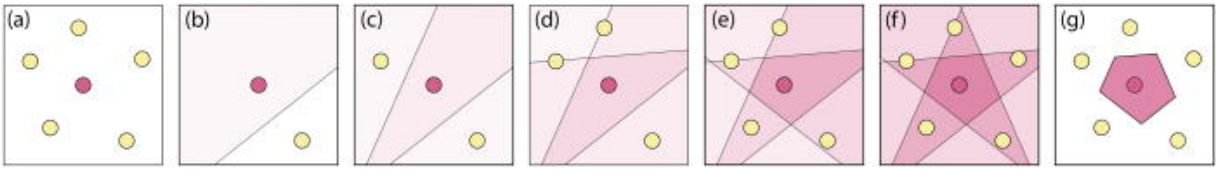

Figure 4.2: Process of construction of a voronoi cell from the intersection of half-spaces. gures (b) to (f) show the process of cutting the space into multiple half-spaces, one half-space at a time. Figure taken from [33]

$$H_{ij} = \{p \in \mathbb{R}^3 \mid d(p, c_i) \leq d(p, c_j)\} \tag{4.2}$$

Every point in R_i is closer to c_i than any other centroid c_k, so it follows that it also has to be in H_{ik} for every $k \neq i$. Conversly, if we have a point $p \in \mathbb{R}^3$ in H_{ik} for every $k \neq i$, it follows that $p \in R_i$. This fact allows us to de ne the voronoi cell R_i as the intersection of a set of hall-spaces:

$$R_i = \bigcap_{j=1}^{n} H_{ij} \tag{4.3}$$

Each half-space is de ned by the plane that is equidistant from both centroids, so this converts the problem of creating a voronoi cell into computing the intersection of multiple planes. Keep in mind that H_{ii} would de ne all the domain, since every point is at the same distance from c_i and c_i, so it does not have any e ect adding it to the intersection. A nice visual demonstration of the construction of a voronoi cell from the intersection of half-spaces is shown in gure ??

Before ending the explanation about cells construction, it is important to keep in mind that a pair of centroids de ne a plane where every point on the plane is equidistant from both centroids. Let us de ne this plane more formally, as we are going to use it later:

$$P_{ij} = P_{ji} = \{p \in \mathbb{R}^3 \mid d(p, c_i) = d(p, c_j)\} \tag{4.4}$$

Neighboring Information

To run the erosion simulation, we will additionally need to create [links] between neighboring cells, which represent the bonds between di erent rocks. Adjacency information is also given by the Voro++ [3] library, but we will provide an overview of the method to compute this information.

We have just commented that in order to generate a voronoi cell, you will need to perform an intersection of halfspaces, which would imply incrementally intersecting your partial solution by one plane at a time. Each time you cut your partial solution by a plane P_{ij}, you can store the ID of the particle j that generated said plane. At the end, just a few planes will remain as faces of the voronoi cell, and those planes will have an ID that indicates which other cell is sharing that speci c face. Using this method, we will compute the neighboring information while we compute the voronoi cells and we can use this information later to generate the links between cells.

Cell Attributes

To end this section about voronoi cells, it is important to highlight what attributes will be stored for each cell, besides the vertices and edges that de ne its shape and its adjacency information.

- Shape Information: For each cell we will store its vertices (with positions), and its faces (with its vertices).
- Additional Face Information: For each face we will also keep its normal, its area, its centroid and the ID of the cell that is on the other side.
- Cell State: We de ne 3 possible states for a cell: Solid, which implies the cell belongs on the terrain (and it has not been removed yet by the erosion simulation); Air, which implies that the cell does not belong to the model; and Core, which makes the cell xed and imposible to remove. A cell can not have two di erent states simultaniously.
- Volume: For some computations we are also interested in keeping track of the volume of a cell.
- Boundary Information: Finally, we will also need to keep track of the solid cells that are on the surface of the model, as they will be in contact with the air and the water in ltration algorithm will start by one of those cells. At the start of the algorithm, exterior cells would be those adjacent to air cells or a wall.

4.2.2 Point Sampling

The rst part of the method consits in sampling multiple points inside the model to construct a voronoi decomposition. In this thesis, and unlike in the base method presented by [32], we are using DEMs to represent and load our terrains, so we are going to make use of that.

A height eld H is basically a 2D grid where each value h_j of the grid; represents the height at one speci c cell $(i; j)$, we can then use the data points that we have to obtain the elevation on a continuous domain using interpolation. Because we already have a de ned grid, we can use it to implement our sampling. The main idea is to use the 2D grid and extend it to a 3D grid by setting a number of vertical samples (which will be a user-di ned parameter) and then sample one point for each element of the grid, as long as the point lies inside the terrain (the 3D grid will represent the bounding box of the terrain, so not every voxel will be inside it).

Now that we have a way to uniformly sample points inside the model, we can extend it and slightly break the regularity by shifting each point by a certain quantity. Considering each point to initially be sampled at the center of its grid cell, we are going to shift each point by strictly less than half the cell size in a random direction. This way we will obtain a sampling scheme between complete random sampling (white noise) and a completely regular sampling, where we guarantee each point to not be too close or too far from each other. Speci cally, from each grid cell center c_{ijk} we create our sample c'_{ijk} using the following equation:

$$c'_{ijk} = c_{ijk} + \begin{bmatrix} r_x s_x \\ r_y s_y \\ r_z s_z \end{bmatrix} \tag{4.5}$$

Where s_x, s_y and s_z are the cell sizes on each direction (we can have rectangular cells) and r_x, r_y and r_z are three di erent scalars sampled uniformly at random such as $\frac{1}{2} < m \quad r_k \quad m < \frac{1}{2}$, where m denotes the maximum ammount of shifting allowed and $k \in \{x; y; z\}$. If m was $\frac{1}{2}$ then samples will be allowed to lie on the boundary of the grid cell, which could result in two samples being at the same place, that is why we set m to be strictly smaller than $\frac{1}{2}$.

for water to be trapped there. Of course, once water is absorbed, we modify the remaining amount of water before jumping to the next link:

$$w^0_{act} = w_{act} \quad w_{abs}(l_i) \tag{5.42}$$

Where w_{act} is the amount of water before the absorption and w^0_{act} is the new amount of water after the absorption. Finally, we also include the possibility of a full absorption event, where w_{abs} becomes w_{act} and all water is absorbed by the link. The probability of this event happening is proportional to the remaining fraction of water $1 \quad \frac{w_{act}}{w_{in}}$.

5.3.2 Updating Links

After a water path crosses a link, the link absorbs part of the water, but it has to be damaged accordingly. The criterion used to modify link's life after water absoprtion in [1] was the followiong:

$$^0_i = \quad _i \quad k_{dmg} \frac{w_{abs}(l_i)}{A_i} \tag{5.43}$$

Where k_{dmg} is a parameter controling how much damage is produced after absorbing a certain quantity per unit area. The idea of this equation works, but we can add the link stresses to extend it. The main idea is that a link under tension lets water pass through more easily, and it is more vulnerable to damage produced by water in ltration, while a link under compression initially produces the opposite e ect, not letting water pass through easily and protecting against erosion; however, if the compression stress is too high, it can produce microfractures on the rock that weaken the link. Regarding shear stress, the e ect is similar to tension; the closer the stress is to the limit, the weaker the link will be.

Putting it all toghether, the damage applied on link l_i will be a ected by modi ers due to the normal and shear stresses:

$$^0_i = \quad ^0_i \quad k_{dmg} \frac{w_{abs}(l_i)}{A_i}(M_n + M_s) \tag{5.44}$$

Where M_n and M_s are the modi ers produced by normal and shear stresses respectively. The normal stress modi er will depend on wether the link is under tension or compression, so we will have 2 equations for M_n that should have the same value when $_n = 0$ (to preserve continuity).

For the tensile part of M_n, when $_n > 0$, we propose an equation based in Gri th's brittle fracture theory [37]. The link weakens exponentially under tension, so we de ne M_n as an exponential:

$$M_n = e^{\ _t(\frac{_n}{T_{max}(D)})^p} \tag{5.45}$$

Where $_t$ controls how sensitive the rock is to tension (how much tension facilitates damage) and p controls the brittleness, higher values of p makes the modi er smaller but with sudden growth when approacing the tension limit. Notice how, regardless of the chosen parameters, $M_n = 1$ when $_n = 0$, as $e^0 = 1$, so the compression criterion that we use also needs to evaluate to 1 at $_n = 0$. In addition to evaluating to 1 under no stress, the modi er should also get smaller when reducing $_n$, as compression provides some protection against damage. However, after $_n$ surpasses a (negative) threshold, microfractures due to compression are produced and the modi er should increase again. In order to represent this phenomenon we can use the following equation using $x = \frac{j \ _n j}{UCS(D)}$: